

L8 Limbajul SQL

Subcereri (Subqueries)

1. Obiective

1. Operatorii pentru mulțimi;
2. Subinterogări și operatorii ANY, ALL, EXISTS
 - 2.1. Subinterogări care returnează un singur rând;
 - 2.2. Subinterogări care returnează mai multe rânduri.
3. Subinterogări imbricate
4. Subinterogări corelate
5. Operații pe tabele ce conțin informații de structură arborescentă

2. Considerații teoretice

2.1 Operatorii pentru mulțimi

Operatorii de mulțimi combină două sau mai multe interogări, efectuând operații specifice multimilor: reuniune, intersecție, diferență.

Acești operatori se mai numesc și operatori verticali deoarece combinarea celor două interogări se face coloană cu coloană. Din acest motiv, numărul total de coloane și tipurile de date ale coloanelor corespondente din cele două interogări trebuie să coincidă:

Există următorii operatori pentru mulțimi:

1. UNION - Returnează rezultatele a două sau mai multe interogări eliminând toate înregistrările duplicate;
2. UNION ALL - Returnează rezultatele a două sau mai multe interogări incluzând înregistrările duplicate;
3. INTERSECT - Returnează toate înregistrările distincte găsite în ambele interogări;
4. MINUS - Returnează toate înregistrările distincte care se găsesc în prima interogare dar nu în a doua interogare.

Pentru o parte din exemple se utilizează și următoarea structură de bază de date:

❖ COLUMN_NAME	❖ DATA_TYPE	❖ NULLABLE	DATA_DEFAULT	❖ COLUMN_ID	❖ COMMENTS
1 ID_PROFESOR	NUMBER(5,0)	No	(null)	1	(null)
2 NUME_PROFESOR	VARCHAR2(20 BYTE)	No	(null)	2	(null)
3 PRENUME_PROFESOR	VARCHAR2(20 BYTE)	No	(null)	3	(null)
4 GRAD_PROFESOR	VARCHAR2(20 BYTE)	No	(null)	4	(null)
5 SALARIU_PROFESOR	NUMBER(5,0)	No	(null)	5	(null)
6 COD_CATEDRA	NUMBER(5,0)	No	(null)	6	(null)

ID_PROFESOR	NUME_PROFESOR	PRENUME_PROFESOR	GRAD_PROFESOR	SALARIU_PROFESOR	COD_CATEDRA
1	100 Nedelcu	Adriana	PROFESOR	60000	10
2	101 Berevoiescu	Claudiu	PROFESOR	55000	10
3	102 Ceapraz	Adina	RECTOR	90000	20
4	103 Bulugea	Bogdan	CONFERENTIAR	40000	15
5	104 Marina	Gabriela	CONFERENTIAR	40000	15
6	105 Geamanu	Alexandru	SL	35000	5
7	106 Oproiu	Florin	SL	35000	5
8	107 Duta	Catalin	SL	35000	5
9	108 Duca	George	LECTOR	30000	1
10	109 Gargut	Mihai	LECTOR	30000	1

Să considerăm de *exemplu* următoarele interogări:

```
SQL> SELECT grad, salariu
      FROM profesor
      WHERE cod_catedra = 10;
```

Rezultat:

GRAD	SALARIU
----	-----
PROF	3000
ASIST	1500
ASIST	1200

```
SQL> SELECT grad, salariu
      FROM profesor
      WHERE cod_catedra = 20;
```

Rezultat:

GRAD	SALARIU
----	-----
PROF	2500
LECT	2200
ASIST	1200

```
SQL> select nume,an
      2 from student
      3 where cods = 1;

NUME                                AN
-----
Popa Marcel                        1
Popescu Ion                        4
Avram Nicolae                      1
```

```
SELECT grad_profesor,salariu_profesor
      FROM bd8
      WHERE cod_catedra=10;
```

GRAD_PROFESOR	SALARIU_PROFESOR
-----	-----
PROFESOR	60000
PROFESOR	55000

GRAD_PROFESOR	SALARIU_PROFESOR
RECTOR	90000

GRAD_PROFESOR	SALARIU_PROFESOR+100
SL	35100
SL	35100
SL	35100

În continuare *exemplificăm* fiecare dintre operatorii pentru mulțimi aplicați acestor interogări:

Exemple:

1.

```
SQL> SELECT grad, salariu
FROM profesor
WHERE cod_catedra = 10
UNION
SELECT grad, salariu
FROM profesor
WHERE cod_catedra = 20
ORDER BY salariu;
```

Rezultat:

GRAD	SALARIU
----	-----
ASIST	1200
ASIST	1500
LECT	2200
PROF	2500
PROF	3000

```
SQL> select nume,an
2  from student
3  where cods=1
4  union
5  select nume,an
6  from student
7  where cods=2
8  order by an;

NUME                                AN
-----
Avram Nicolae                       1
Popa Marcel                         1
Popescu Gina                        3
Ionescu Mariana                     4
Popescu Ion                         4

SQL>
```

```
SELECT grad_profesor,salariu_profesor
FROM bd8
WHERE cod_catedra=10
UNION
SELECT grad_profesor,salariu_profesor
FROM bd8
WHERE cod_catedra=20
ORDER BY salariu_profesor;
```

GRAD_PROFESOR	SALARIU_PROFESOR
-----	-----
PROFESOR	55000
PROFESOR	60000
RECTOR	90000

2.

```
SQL> SELECT grad, salariu
FROM profesor
WHERE cod_catedra = 10
UNION ALL
SELECT grad, salariu
FROM profesor
WHERE cod_catedra = 20;
```

Rezultat:

GRAD	SALARIU
----	-----
PROF	3000
ASIST	1500
ASIST	1200
PROF	2500
LECT	2200
ASIST	1200

```
SQL> select nume,an
2  from student
3  where cods=1
4  union all
5  select nume,an
6  from student
7  where cods=2;
```

NUME	AN
-----	-----
Popa Marcel	1
Popescu Ion	4
Avram Nicolae	1
Ionescu Mariana	4
Popescu Gina	3

```
SELECT grad_profesor,salariu_profesor
FROM bd8
WHERE cod_catedra=10
UNION ALL
SELECT grad_profesor,salariu_profesor
FROM bd8
WHERE cod_catedra=1
ORDER BY salariu_profesor;
```

GRAD_PROFESOR	SALARIU_PROFESOR
-----	-----
LECTOR	30000
LECTOR	30000
PROFESOR	55000
PROFESOR	60000

3.

```
SQL> SELECT grad, salariu
      FROM profesor
      WHERE cod_catedra = 10
      INTERSECT
      SELECT grad, salariu
      FROM profesor
      WHERE cod_catedra = 20;
```

Rezultat:

GRAD	SALARIU
----	-----
ASIST	1200

```
SQL> select nume,an
2  from student
3  where cods=1
4  INTERSECT
5  select nume,an
6  from student
7  where cods=1;

NUME                                AN
-----
Avram Nicolae                       1
Popa Marcel                         1
Popescu Ion                         4
SQL>
```

```
SELECT grad_profesor,salariu_profesor
FROM bd8
WHERE cod_catedra=10
INTERSECT
SELECT grad_profesor,salariu_profesor
FROM BD8
WHERE cod_catedra=1||15||20||5
ORDER BY salariu_profesor
```

GRAD_PROFESOR	SALARIU_PROFESOR
-----	-----
PROFESOR	55000
PROFESOR	60000

4.

```
SQL> SELECT grad, salariu
      FROM profesor
      WHERE cod catedra = 10
      MINUS
      SELECT grad, salariu
      FROM profesor
      WHERE cod catedra = 20;
```

Rezultat:

GRAD	SALARIU
----	-----

ASIST	1500
PROF	3000

```
SQL> select nume,an
2  from student
3  where cods=1
4  MINUS
5  select nume,an
6  from student
7  where cods=2;
```

NUME	AN
Avram Nicolae	1
Popa Marcel	1
Popescu Ion	4

```
SELECT grad_profesor,salariu_profesor
FROM bd8
WHERE cod_catedra=20
MINUS
SELECT grad_profesor,salariu_profesor
FROM bd8
WHERE cod_catedra=15
ORDER BY salariu_profesor;
```

GRAD_PROFESOR	SALARIU_PROFESOR
PROFESOR	55000
PROFESOR	60000
GRAD_PROFESOR	SALARIU_PROFESOR
RECTOR	90000

```
SELECT salariu_profesor
FROM bd8
MINUS
SELECT id_profesor
FROM bd8;
```

SALARIU_PROFESOR
30000
35000
40000
55000
60000
90000

Există următoarele reguli de folosire a operatorilor pentru mulțimi:

- interogările trebuie să conțină același număr de coloane;
- coloanele corespondente trebuie să aibă același tip de dată;
- în rezultat vor apărea numele coloanelor din prima interogare, nu cele din a doua interogare chiar dacă aceasta folosește alias-uri.

Exemplu:

```
SQL> SELECT cod
FROM profesor
MINUS
SELECT sef
FROM profesor;
```

Rezultat:

```
COD
---
101
104
105
106
```

- clauza **ORDER BY** poate fi folosită o singură dată într-o interogare care folosește operatori de mulțimi; atunci când se folosește, ea trebuie poziționată la sfârșitul comenzii;

Exemplu:

```
SQL> SELECT grad, salariu
      FROM profesor
      WHERE cod_catedra = 10
      UNION
      SELECT grad, salariu
      FROM profesor
      WHERE cod_catedra = 20
      ORDER BY 2;
```

Rezultat:

GRAD	SALARIU
----	-----
ASIST	1200
ASIST	1500
LECT	2200
PROF	2500
PROF	3000

- operatorii pentru mulțimi pot fi utilizați în subinterogări;
- pentru a modifica ordinea de execuție este posibilă utilizarea parantezelor.

Exemple:

```
SQL> SELECT grad
      FROM profesor
      WHERE cod_catedra = 10
      INTERSECT
      SELECT grad
      FROM profesor
      WHERE cod_catedra = 20
      UNION
      SELECT grad
      FROM profesor
      WHERE cod_catedra = 30;
```

Rezultat:

```
GRAD
----
ASIST
CONF
PROF
```

```
SQL> SELECT grad
      FROM profesor
      WHERE cod_catedra = 10
      INTERSECT
      (SELECT grad
       FROM profesor
       WHERE cod_catedra = 20
      UNION
      SELECT grad
       FROM profesor
       WHERE cod_catedra = 30);
```

Rezultat:

```
GRAD
----
ASIST
PROF
```

```
SELECT grad_profesor
FROM bd8
WHERE cod_catedra = 10
INTERSECT
(SELECT grad_profesor
FROM bd8
WHERE cod_catedra= 20
UNION
SELECT grad_profesor
FROM bd8
WHERE cod_catedra= 15);
```

```
SQL> SELECT GRAD_PROFESOR
2  FROM BD8
3  WHERE COD_CATEDRA = 10
4  INTERSECT
5  (SELECT GRAD_PROFESOR
6  FROM BD8
7  WHERE COD_CATEDRA= 20
8  UNION
9  SELECT GRAD_PROFESOR
10 FROM BD8
11 WHERE COD_CATEDRA= 15);

no rows selected
```

2.2 Subinterogări și operatorii ANY, ALL, EXISTS

O **subinterogare** este o comandă SELECT inclusă în altă comandă SELECT. Rezultatele subinterogării sunt transmise celeilalte interogări și pot apărea în cadrul clauzelor WHERE, HAVING sau FROM. Subinterogările sunt utile pentru a scrie interogări bazate pe o condiție în care valoarea de comparație este necunoscută. Această valoare poate fi aflată folosind o subinterogare.

De exemplu:

```
SELECT coloane
```



```
FROM tabel
WHERE coloana = (SELECT coloane
                  FROM tabel
                  WHERE conditie) .
```

Subinterogarea, denumită și interogare interioară (inner query), generează valorile pentru condiția de căutare a instrucțiunii SELECT care o conține, denumită interogare exterioară (outer query). Instrucțiunea SELECT exterioară depinde de valorile generate de către interogarea interioară. În general, interogarea interioară se execută prima și rezultatul acesteia este utilizat în interogarea exterioară. Rezultatul interogării exterioare depinde de numărul valorilor returnate de către interogarea interioară.

În acest sens, putem distinge:

1. Subinterogări care returnează un singur rând;

```
SELECT nume_profesor, prenume_profesor, salariu_profesor
FROM bd8
WHERE salariu_profesor = (SELECT MIN (salariu_profesor)
FROM bd8) ;
```

NUME_PROFESOR	PRENUME_PROFESOR	SALARIU_PROFESOR
Duca	George	30000
Gargut	Mihai	30000

2. Subinterogări care returnează mai multe rânduri.

Din punct de vedere al ordinii de evaluare a interogărilor putem clasifica subinterogările în:

1. **Subinterogări simple** - în care interogarea interioară este evaluată prima, independent de interogarea exterioară (interogarea interioară se execută o singură dată);
2. **Subinterogări corelate** - în care valorile returnate de interogarea interioară depind de valorile returnate de interogarea exterioară (interogarea interioară este evaluată pentru fiecare înregistrare a interogării exterioare).

Subinterogările sunt îndeosebi utilizate atunci când se dorește ca o interogare să regăsească înregistrări dintr-o tabelă care îndeplinesc o condiție ce depinde la rândul ei de valori din aceeași tabelă.

```
SELECT nume_profesor, prenume_profesor, salariu_profesor
FROM bd8
WHERE salariu_profesor=35000;
```

NUME_PROFESOR	PRENUME_PROFESOR	SALARIU_PROFESOR
Geamanu	Alexandru	35000
Oproiu	Florin	35000
Duta	Catalin	35000

Observație: Clauza ORDER BY nu poate fi utilizată într-o subinterogare. Regula este că poate exista doar o singură clauză ORDER BY pentru o comandă SELECT și, dacă este specificată, trebuie să fie ultima clauză din comanda SELECT. Prin urmare, clauza ORDER BY nu poate fi specificată decât în interogarea cea mai din exterior.

2.2.1 Subinterogări care returnează un singur rând

În acest caz condiția, din clauza WHERE sau HAVING a interogării exterioare utilizează operatorii: =, <, <=, >, >=, <> care operează asupra unei subinterogări ce returnează o singură valoare. Interogarea interioară poate conține condiții complexe formate prin utilizarea condițiilor multiple de interogare cu ajutorul operatorilor AND și OR sau prin utilizarea funcțiilor agregat.

Următoarea interogare selectează cadrele didactice care au salariul cel mai mic. Salariul minim este determinat de o subinterogare ce returnează o singură valoare.

```
SQL> SELECT nume, prenume, salariu
      FROM profesor
      WHERE salariu = (SELECT MIN (salariu)
                      FROM profesor);
```

Rezultat:

NUME	PRENUME	SALARIU
-----	-----	-----
VOINEA	MIRCEA	1200
STANESCU	MARIA	1200

Procesul de evaluare al acestei interogări se desfășoară astfel:

- Se evaluează în primul rând interogarea interioară:
 - Valoarea obținută este MIN (salariu) = 1 200
- Rezultatul evaluării interogării interioare devine condiție de căutare pentru interogarea exterioară și anume:

```
SQL> SELECT nume, prenume, salariu
      FROM profesor
      WHERE salariu = 1200;
```

În cazul în care interogarea interioară nu întoarce nici o înregistrare, interogarea exterioară nu va selecta la rândul ei nici o înregistrare.

```
SQL> select nume,an,grupa
      2  from student
      3  where an=(select min(an) from student) ;
```

NUME	AN
-----	-----
GRUPA	

Popa Marcel	1
114A	
Avram Nicolae	1
115A	

```
SQL>
```

Observație: Dacă se utilizează operatorii: =, <, <=, >, >=, <> în condiția interogării exterioare, atunci interogarea interioară trebuie în mod obligatoriu să returneze o singură valoare. În caz contrar va apărea un mesaj de eroare, ca în exemplul următor:

```
SQL> SELECT nume, prenume, salariu
      FROM profesor
      WHERE salariu = (SELECT MIN (salariu)
                      FROM profesor
                      GROUP BY grad);
```

ORA-01427: single-row subquery returns more than one row

```
SQL> SELECT grad
FROM profesor
GROUP BY grad
HAVING MIN(salariu) > (SELECT AVG(salariu)
                        FROM profesor);
```

GRAD

CONT
LECT
PROF

```
SQL> select grupa
      2 from student
      3 group by grupa
      4 having min (an)> (select avg (an) from student);

GRUPA
-----
121B
116C
```

```
SELECT grad_profesor
FROM bd8
GROUP BY grad_profesor
HAVING MIN(salariu_profesor) > (SELECT AVG(salariu_profesor)
FROM bd8);
```

GRAD_PROFESOR

PROFESOR
RECTOR

În cazul când interogarea întoarce mai multe rânduri nu mai este posibilă folosirea operatorilor de comparație. În locul acestora se folosește operatorul IN, care așteaptă o listă de valori și nu doar una.

Următoarea interogare selectează pentru fiecare grad didactic acele persoane care au salariul minim. Salariul minim pentru fiecare grad didactic este aflat printr-o subinterogare, care, evident, va întoarce mai multe rânduri:

```
SQL> SELECT nume, salariu, grad  
      FROM profesor  
      WHERE (salariu, grad) IN  
                                (SELECT MIN (salariu), grad  
                                 FROM profesor  
                                 GROUP BY grad)  
  
      ORDER BY salariu;
```

Rezultat:

NUME	SALARIU	GRAD
----	-----	----
VOINEA	1200	ASIST
STANESCU	1200	ASIST
ALBU	2200	LECT
MARIN	2500	PROF
GEORGESCU	2800	CONF

```
SQL> select nume, an, grupa
2   from student
3  where(an,grupa) IN (select min(an), grupa from student group by grupa)
4  order by an;

NUME                                AN
-----
GRUPA
-----
Popa Marcel                        1
114A
Avram Nicolae                     1
115A
Popescu Ion                       4
1218

NUME                                AN
-----
GRUPA
-----
Ionescu Mariana                   4
116C
```

Observație: Spre deosebire de celelalte interogări de până acum, interogarea de mai sus compară perechi de coloane. În acest caz trebuie respectate următoarele reguli:

- coloanele din dreapta condiției de căutare sunt în paranteze și fiecare coloană este separată prin virgulă;
- coloanele returnate de interogarea interioară trebuie să se potrivească ca număr și tip cu coloanele cu care sunt comparate în interogarea exterioară; în plus, ele trebuie să fie în aceeași ordine cu coloanele cu care sunt comparate.

Alături de operatorul IN, o subinterogare care returnează mai multe rânduri poate folosi operatorii ANY, ALL sau EXISTS. Operatorii ANY și ALL sunt prezentați în continuare, iar operatorul EXISTS va fi prezentat în secțiunea 'Subinterogări corelate'.

Operatorii ANY și ALL sunt folosiți în mod obligatoriu în combinație cu operatorii relaționali =, !=, <, >, <=, >; operatorii IN și EXISTS nu pot fi folosiți în combinație cu operatorii relaționali, dar pot fi utilizați cu operatorul NOT, pentru negarea expresiei.

Operatorul ANY

Operatorul **ANY** (sau sinonimul sau SOME) este folosit pentru a compara o valoare cu oricare dintre valorile returnate de o subinterogare. Pentru a intelege modul de folosire a acestui operator sa consideram urmatorul exemplu ce afiseaza cadrele didactice ce castiga mai mult decat profesorii care au cel mai mic salariu:

```
SQL> SELECT nume, salariu, grad
FROM profesor
WHERE salariu > ANY (SELECT DISTINCT salariu
                     FROM profesor
                     WHERE grad='PROF');
```

Rezultat:

NUME	SALARIU	GRAD
----	-----	----
GHEORGHIU	3000	PROF
GEORGESCU	2800	CONF

Interogarea de mai sus este evaluată astfel:

- dacă salariul unui cadru didactic este mai mare decât cel puțin unul din salariile returnate de interogarea interioară, acea înregistrare este inclusă în rezultat.
- Cu alte cuvinte, >ANY înseamnă mai mare decât minimul dintre valorile returnate de interogarea interioară, <ANY înseamnă mai mic ca maximumul, iar =ANY este echivalent cu operatorul IN.

```
SQL> select nume, an, grupa
  2  from student
  3  where an>any (select distinct an from student where grupa='114A');

NUME                                AN
-----
GRUPA
-----
Popescu Ion                        4
121B
Ionescu Mariana                    4
116C
Popescu Gina                       3
114A
```

Observație: Opțiunea DISTINCT este folosită frecvent atunci când se folosește operatorul ANY pentru a preveni selectarea de mai multe ori a unor înregistrări.

Operatorul ALL

Operatorul ALL este folosit pentru a compara o valoare cu toate valorile returnate de o subinterogare. Considerăm următorul exemplu ce afișează cadrele didactice care câștigă mai mult decât asistenții cu salariul cel mai mare:

```
SQL> SELECT nume, salariu, grad
      FROM profesor
      WHERE salariu > ALL (SELECT DISTINCT salariu
                           FROM profesor
                           WHERE grad='ASIST');
```

Rezultat:

NUME	SALARIU	GRAD
----	-----	----
GHEORGHIU	3000	PROF
MARIN	2500	PROF
GEORGESCU	2800	CONF
ALBU	2200	LECT

Interogarea de mai sus este evaluată astfel: dacă salariul unui cadru didactic este mai mare decât toate valorile returnate de interogarea interioară, acea înregistrare este inclusă în rezultat. Cu alte cuvinte, >ALL înseamnă mai mare ca maximumul dintre valorile returnate de interogarea interioară iar <ALL înseamnă mai mic ca minimumul dintre acestea.

```
SQL> select nume,an,grupa
2   from student
3   where an> all(select distinct an from student where grupa='114A');

NUME                                     AN
-----
GRUPA
-----
Popescu Ion                             4
121B
Ionescu Mariana                         4
116C
```

Observație: Operatorul ALL nu poate fi utilizat cu operatorul = deoarece interogarea nu va întoarce nici un rezultat cu excepția cazului în care toate valorile sunt egale, situație care nu ar avea sens.

2.3 Subinterogări imbricate

Subinterogările pot fi imbricate (utilizate cu alte subinterogări) până la 255 de nivele, indiferent de numărul de valori returnate de fiecare subinterogare. Pentru a selecta cadrele didactice care au salariul mai mare decât cel mai mare salariu al cadrelor didactice care aparțin secției de Electronică, vom folosi următoarea interogare:

```
SQL> SELECT nume, prenume, salariu
FROM profesor
WHERE salariu > (SELECT MAX(salariu)
                  FROM profesor
                  WHERE cod_catedra=(SELECT cod_catedra
                                      FROM catedra
                                      WHERE nume= 'ELECTRONICA'));
```

2.4 Subinterogări corelate

În exemplele considerate până acum interogarea interioară era evaluată prima, după care valoarea sau valorile rezultate erau utilizate de către interogarea exterioară. Subinterogările de acest tip sunt numite subinterogări simple. O altă formă de subinterogare o reprezintă interogarea corelată, caz în care interogarea exterioară transmite repetat câte o înregistrare pentru interogarea interioară. Interogarea interioară este evaluată de fiecare dată când este transmisă o înregistrare din interogarea exterioară, care se mai numește și înregistrare candidată. Subinterogarea corelată poate fi identificată prin faptul că interogarea interioară nu se poate executa independent ci depinde de valoarea transmisă de către interogarea exterioară. Dacă ambele interogări accesează aceeași tabelă, trebuie asigurate alias-uri pentru fiecare referire la tabela respectivă.

Subinterogările corelate reprezintă o cale de a accesa fiecare înregistrare din tabel și de a compara anumite valori ale acesteia cu valori ce depind tot de ea.

Evaluarea unei subinterogări corelate se execută în următorii pași:

1. Interogarea exterioară trimite o înregistrare candidată către interogarea interioară;
2. Interogarea interioară se execută în funcție de valorile înregistrării candidate;
3. Valorile rezultate din interogarea interioară sunt utilizate pentru a determina dacă înregistrarea candidată va fi sau nu inclusă în rezultat;
4. Se repetă procedeul începând cu pasul 1 până când nu mai există înregistrări candidate.

De exemplu pentru a regăsi cadrele didactice care câștigă mai mult decât salariul mediu din propria catedră, putem folosi următoarea interogare corelată:

```
SQL> SELECT nume, prenume, salariu
FROM profesor p
WHERE salariu > (SELECT AVG(salariu)
                  FROM profesor s
```

```
WHERE s.cod_catedra = p.cod_catedra);
```

Rezultat:

NUME	PRENUME	SALARIU
----	-----	-----
GHEORGHIU	STEFAN	3000
MARIN	VLAD	2500
ALBU	GHEORGHE	2200

În exemplul de mai sus coloana interogării exterioare care se folosește în interogarea interioară este p. cod_catedra. Deoarece p. cod_catedra poate avea o valoare diferită pentru fiecare înregistrare, interogarea interioară se execută pentru fiecare înregistrare candidată transmisă de interogarea exterioară.

Atunci când folosim subinterogări corelate împreună cu clauza HAVING, coloanele utilizate în această clauză trebuie să se regăsească în clauza GROUP BY. În caz contrar, va fi generat un mesaj de eroare datorat faptului că nu se poate face comparație decât cu o expresie de grup. De exemplu, următoarea interogare este corectă, ea selectând gradele didactice pentru care media salariului este mai mare decât maximul primei pentru același grad:

```
SQL> SELECT grad
      FROM profesor p
      GROUP BY grad
      HAVING AVG (salariu) > (SELECT MAX(prima)
                             FROM profesor
                             WHERE grad = p.grad);
```

Rezultat:

grad

ASIST
CONF

Operatorul EXISTS

Operatorul EXISTS verifică dacă, pentru fiecare înregistrare transmisă de interogarea exterioară, există sau nu înregistrări care satisfac condiția interogării interioare, returnând interogării exterioare valoarea True sau False. Cu alte cuvinte, operatorul EXISTS cere în mod obligatoriu corelarea interogării interioare cu interogarea exterioară. Datorită faptului că operatorul EXISTS verifică doar existența rândurilor selectate și nu ia în considerare numărul sau valorile atributelor selectate, în subinterogare poate fi specificat orice număr de attribute; în particular, poate fi folosită o constantă și chiar simbolul * (deși acest lucru nu este recomandabil din punct de vedere al eficienței). De altfel, EXISTS este singurul operator care permite acest lucru.

Următoarea interogare selectează toate cadrele didactice care au măcar un subordonat:

```
SQL> SELECT cod, nume, prenume, grad
      FROM profesor p
      WHERE EXISTS
            (SELECT '1'
             FROM profesor
             WHERE profesor.sef = p.cod)
      ORDER BY cod;
```

Rezultat:

cod	nume	prenume	grad
100	GHEORGHIU	STEFAN	PROF
102	GEORGESCU	CRISTIANA	CONF
103	IONESCU	VERONICA	ASIST

La fel ca și operatorul IN, operatorul EXISTS poate fi negat, luând forma NOT EXISTS. Totuși, o remarcă foarte importantă este faptul că pentru subinterogări, NOT IN nu este la fel de eficient ca NOT EXISTS. Astfel dacă în lista de valori transmisă operatorului NOT IN exista una sau mai multe valori Null, atunci condiția va lua valoarea de adevăr False, indiferent de celelalte valori din listă.

De exemplu, următoarea interogare încearcă să returneze toate cadrele didactice care nu au nici un subaltern:

```
SQL> SELECT nume, grad
      FROM profesor
      WHERE cod NOT IN (SELECT sef
                        FROM profesor);
```

Această interogare nu va întoarce nici o înregistrare deoarece coloana sef conține și valoarea Null. Pentru a obține rezultatul corect trebuie să folosim următoarea interogare:

```
SQL> SELECT nume, grad
      FROM profesor p
      WHERE NOT EXISTS (SELECT '1'
                        FROM profesor
                        WHERE sef=p.cod);
```

Rezultat:

nume	grad
MARIN	PROF
ALBU	LECT
VOINEA	ASIST
STANESCU	ASIST

În general, operatorul EXISTS se folosește în cazul subinterogărilor corelate și este câteodată cel mai eficient mod de a realiza anumite interogări. Performanța interogărilor depinde de folosirea indecșilor, de numărul rândurilor returnate, de dimensiunea tabelului și de necesitatea creării tabelului temporar pentru evaluarea rezultatelor intermediare. Tabelele temporare generate de Oracle nu sunt indexate, iar acest lucru poate degrada performanța subinterogărilor dacă se folosesc operatorii IN, ANY sau ALL.

Subinterogările mai pot apărea și în alte comenzi SQL cum ar fi: UPDATE, DELETE, INSERT și CREATE TABLE.

Așa cum am văzut, există în principal două moduri de realizare a interogărilor ce folosesc date din mai multe tabele: joncțiuni și subinterogări. Joncțiunile reprezintă forma de interogare relațională (în care sarcina găsirii drumului de acces la informație revine SGRD-ului) iar subinterogările forma procedurală (în care trebuie indicat drumul de acces la informație). Fiecare dintre aceste forme are avantajele sale, depinzând de cazul specific în care se aplica.

2.5 Operații pe tabele ce conțin informații do structura arborescentă

O bază de date relațională nu poate stoca înregistrări în mod ierarhic, dar la nivelul înregistrării pot exista informații care determină o relație ierarhică între înregistrări. SQL permite afișarea rândurilor dintr-o tabelă ținând cont de relațiile ierarhice care apar între rândurile tabelului. Parcurgerea în mod ierarhic a informațiilor se poate face doar la nivelul unei singure tabele. Operația se realizează cu ajutorul clauzelor START WITH și CONNECT BY din comanda SELECT.

De exemplu, în tabela profesor există o relație ierarhică între înregistrări datorată valorilor din coloanele cod și sef. Fiecare înregistrare aferentă unui cadru didactic conține în coloana sef codul persoanei căreia îi este direct subordonat. Pentru a obține o situație ce conține nivelele ierarhice, vom folosi următoarea interogare:

```
SQL> SELECT LEVEL, nume, prenume, grad
      FROM profesor
      CONNECT BY PRIOR cod=sef
      START WITH sef IS NULL;
```

Rezultat:

LEVEL	NUME	PRENUME	GRAD
-----	-----	-----	----
1	GHEORGHIU	STEFAN	PROF
2	MARIN	VLAD	PROF
2	GEORGESCU	CRISTIANA	CONF
3	IONESCU	VERONICA	ASIST
4	STANESCU	MARIA	ASIST
2	ALBU	GHEORGHE	LECT
2	VOINEA	MIRCEA	ASIST

Explicarea sintaxei și a regulilor de funcționare pentru exemplul de mai sus:

- Clauza standard SELECT poate conține pseudo-coloana LEVEL ce indică nivelul înregistrării în arbore (cât de departe este de nodul radacină). Astfel, nodul rădăcina are nivelul 1, fiii acestuia au nivelul 2, ș.a.m.d.;
- În clauza FROM nu se poate specifica decât o tabelă;
- Clauza WHERE poate apărea în interogare pentru a restricționa vizitarea nodurilor (înregistrărilor) din cadrul arborelui;
- Clauza CONNECT BY specifică coloanele prin care se realizează relația ierarhică; acesta este clauza cea mai importantă pentru parcurgerea arborelui și este obligatorie;
- Operatorul PRIOR stabilește direcția în care este parcurs arborele. Dacă clauza apare înainte de atributul cod, arborele este parcurs de sus în jos, iar dacă apare înainte de atributul sef arborele este parcurs de jos în sus;
- Clauza START WITH specifică nodul (înregistrarea) de început a arborelui. Ca punct de start nu se poate specifica un anumit nivel (LEVEL), ci trebuie specificată

valoarea; această clauză este opțională, dacă ea lipsește, pentru fiecare înregistrare se va parcurge arborele care are ca rădăcină această înregistrare.

În sintaxa interogării de mai sus, pentru a ordona înregistrările returnate, poate apărea clauza OROER BY, dar este recomandabil să nu o folosim deoarece ordinea implicită de parcurgere a arborelui va fi distrusă.

Pentru a elimina doar un anumit nod din arbore putem folosi clauza WHERE, iar pentru a elimina o întreagă ramură dintr-un arbore (o anumită înregistrare împreună cu fiii acesteia) folosim o condiție compusă în clauza CONNECT BY.

Următorul *exemplu* elimină doar înregistrarea cu numele 'GEORGESCU', dar nu și fiii acesteia:

```
SQL> SELECT LEVEL, nume, prenume, grad
      FROM profesor
      WHERE nume != 'GEORGESCU'
      CONNECT BY PRIOR cod=sef
      START WITH sef IS NULL;
```

Rezultat:

LEVEL	NUME	PRENUME	GRAD
-----	----	-----	----
1	GHEORGHIU	STEFAN	PROF
2	MARIN	VLAD	PROF
3	IONESCU	VERONICA	ASIST
4	STANESCU	MARIA	ASIST
2	ALBU	GHEORGHE	LECT
2	VOINEA	MIRCEA	ASIST

Pentru a elimina toată ramura care conține înregistrarea cu numele 'GEORGESCU' și înregistrările pentru subordonații acesteia se folosește următoarea interogare:

```
SQL> SELECT LEVEL, nume, prenume, grad
      FROM profesor
      CONNECT BY PRIOR cod=sef AND nume != 'GEORGESCU'
      START WITH sef IS NULL;
```

Rezultat:

LEVEL	NUME	PRENUME	GRAD
-----	----	-----	----
1	GHEORGHIU	STEFAN	PROF
2	MARIN	VLAD	PROF
2	ALBU	GHEORGHE	LECT
2	VOINEA	MIRCEA	ASIST

O subcerere este o comandă SELECT încapsulată într-o clauză a altei instrucțiuni SQL, numită instrucțiune „părinte“. Utilizând subcereri, se pot construi interogări complexe pe baza unor instrucțiuni simple. Subcererile mai sunt numite instrucțiuni SELECT imbricate sau interioare.

Subcererea returnează o valoare care este utilizată de către instrucțiunea „părinte“. Utilizarea unei subcereri este echivalentă cu efectuarea a două cereri secvențiale și utilizarea rezultatului cererii interne ca valoare de căutare în cererea externă (principală).

Subcererile sunt de 2 feluri:

1. **necorelate**, de forma :

```
SELECT  lista_select
FROM    nume_tabel
WHERE   expresie_operator (SELECT      lista_select
                              FROM      nume_tabel);
```

- cererea internă este executată prima și determină o valoare (sau o mulțime de valori);
- cererea externă se execută o singură dată, utilizând valorile returnate de cererea internă.

2. **corelate**, de forma:

```
SELECT nume_coloană_1[, nume_coloană_2 ...]
FROM   nume_tabel_1 extern
WHERE  expresie_operator
      (SELECT  nume_coloană_1 [, nume_coloană_2 ...]
      FROM nume_tabel_2
      WHERE    expresie_1 = extern.expresie_2);
```

- cererea externă determină o linie candidat;
- cererea internă este executată utilizând valoarea liniei candidat;
- valorile rezultate din cererea internă sunt utilizate pentru calificarea sau descalificarea liniei candidat;
- pașii precedenți se repetă până când nu mai există linii candidat.

Observație: **operator** poate fi:

- single-row operator (>, =, >=, <, <>, <=), care poate fi utilizat dacă subcererea returnează o singură linie;
- multiple-row operator (IN, ANY, ALL), care poate fi folosit dacă subcererea returnează mai mult de o linie.
- Operatorul NOT poate fi utilizat în combinație cu IN, ANY și ALL.

3. Desfășurarea lucrării

Exerciții - subcereri necorelate

1. Folosind subcereri, să se afișeze numele și data angajării pentru salariații care au fost angajați după angajatul cu numele Gates.

```
SELECT last_name, hire_date
FROM   employees
WHERE  hire_date > (SELECT hire_date
                    FROM   employees
                    WHERE  INITCAP(last_name)='Gates');
```

2. Folosind subcereri, scrieți o cerere pentru a afișa numele și salariul pentru toți colegii (din același departament) lui Gates. Se va exclude Gates.

```
SELECT last_name, salary
FROM   employees
```

```

WHERE department_id IN (SELECT department_id
                        FROM employees
                        WHERE LOWER(last_name)='gates')
AND LOWER(last_name) <> 'gates';

```

? Se putea pune "=>" în loc de "IN"? În care caz nu se poate face această înlocuire?

3. Folosind subcereri, să se afișeze numele și salariul angajaților conduși direct de președintele companiei (acesta este considerat angajatul care nu are manager).

```

SELECT last_name, salary
FROM employees
WHERE manager_id = (SELECT employee_id
                   FROM employees
                   WHERE manager_id IS NULL);

```

4. Scrieti o cerere pentru a afișa numele, codul departamentului si salariul angajatilor al caror număr de departament si salariu coincid cu numarul departamentului si salariul unui angajat care castiga comision.

```

SELECT last_name, department_id, salary
FROM employees
WHERE (department_id, salary) IN (SELECT department_id, salary
                                FROM employees
                                WHERE commission_pct IS NOT NULL

```

5. Scrieti o cerere pentru a afisa angajatii care castiga mai mult decat oricare functionar (job-ul conține șirul "CLERK"). Sortati rezultatele dupa salariu, in ordine descrescatoare.

```

SELECT last_name, salary
FROM employees
WHERE salary > ALL(SELECT salary
                  FROM employees
                  WHERE LOWER(job_id) LIKE '%clerk%')
ORDER BY 2 DESC;

```

? Ce rezultat este returnat dacă se înlocuiește "ALL" cu "ANY"?

6. Scrieți o cerere pentru a afișa numele, numele departamentului și salariul angajaților care nu câștigă comision, dar al căror șef direct coincide cu șeful unui angajat care câștigă comision.

```

SELECT e.last_name, d.department_name, e.salary
FROM employees e, departments d
WHERE e.department_id = d.department_id
AND commission_pct IS NULL
AND e.manager_id IN (SELECT manager_id
                    FROM employees
                    WHERE commission_pct IS NOT NULL);

```

7. Sa se afiseze numele, departamentul, salariul și job-ul tuturor angajatilor al caror salariu si comision coincid cu salariul si comisionul unui angajat din Oxford.

```

SELECT e.last_name, d.department_name, e.salary, j.job_title

```

```

FROM employees e, departments d, jobs j
WHERE e.department_id = d.department_id
AND e.job_id = j.job_id
AND (e.salary, NVL(e.commission_pct, 0)) IN
      (SELECT salary, NVL(commission_pct,0)
       FROM employees
       WHERE department_id IN
            (SELECT department_id
             FROM departments
             WHERE location_id IN
                  (SELECT location_id
                   FROM locations
                   WHERE LOWER(city)='oxford')));

```

8. Să se afișeze numele angajaților, codul departamentului și codul job-ului salariaților al căror departament se află în Toronto.

```

SELECT last_name, job_id, department_id
FROM employees
WHERE department_id IN (SELECT department_id
                        FROM departments
                        WHERE location_id = (SELECT location_id
                                           FROM locations
                                           WHERE city = 'Toronto'));

```

9. Sa se afișeze codul, numele și salariul tuturor angajaților care câștigă mai mult decât salariul mediu pentru job-ul corespunzător și lucrează într-un departament cu cel puțin unul din angajații al căror nume conține litera “t”.